From Symmetric to Directed Membership: How the Attractive/Repulsive Forces Were Fixed

1 From Symmetric to Directed Membership: How the Attractive/Repulsive Forces Were Fixed

1.1 Where this comes from: UMAP is cross-entropy minimization

UMAP does not invent its attractive and repulsive forces by hand. It builds a *target* membership $\mu(i,j) \in [0,1]$ for every pair of points from the high-dimensional data (how strongly j should be considered a neighbour of i), and a *model* membership

$$\nu(i,j) = \frac{1}{1 + a\,\rho(i,j)^{2b}}$$

from the current low-dimensional embedding, where $\rho(i,j)$ is the distance between y_i and y_j. It then minimizes the cross-entropy between the two:

$$CE = \sum_{i,j} \underbrace{\mu(i,j) \log \frac{\mu(i,j)}{\nu(i,j)}}_{\text{attractive term}} + \underbrace{\big(1-\mu(i,j)\big) \log \frac{1-\mu(i,j)}{1-\nu(i,j)}}_{\text{repulsive term}}.$$

Taking the gradient of this loss with respect to the embedding positions is exactly where the attractive coefficient $\text{attr\_coeff}(\rho)$ and repulsive coefficient $\text{rep\_coeff}(\rho)$ come from. Concretely, each pair's contribution to the force is

$$\mu(i,j) \cdot \text{attr\_coeff}(\rho) \;-\; \big(1-\mu(i,j)\big) \cdot \text{rep\_coeff}(\rho).$$

So $\mu(i,j)$ is not a cosmetic weight – it is literally the coefficient the cross-entropy loss assigns to the attractive term for that pair. If $\mu(i,j)$ is high, j pulls i strongly; if it is low, j mostly pushes i away.

1.2 The problem: our data is directional, but μ was forced symmetric

In vanilla UMAP, each point i first computes its own, one-sided view of the membership from its own k nearest neighbours:

$$A(i,j) = \exp\!\left(-\frac{\max(0,\, D(i,j) - \rho_i)}{\sigma_i}\right),$$

using only row i of the distance matrix D. This A is not symmetric in general: $A(i,j) \neq A(j,i)$, because i and j may disagree on how close they are to each other. Vanilla UMAP then forces a single, shared value per edge by symmetrizing it with the probabilistic t-conorm,

$$\mu_{\text{sym}} = A + A^\top - A \cdot A^\top,$$

and uses μ_{sym} everywhere afterwards: in the cross-entropy weighting above, and in building the initial layout (the eigendecomposition used for spectral initialization requires a symmetric matrix).

This project's distance matrix D_{asym} is asymmetric on purpose – it encodes a Randers (directional) field. The reconstructed distance in the embedding, $\rho_r(i \to j) = \|y_i - y_j\| + b_i \cdot (y_j - y_i)$, is *also* asymmetric on purpose, by the same Randers substitution. So we had an asymmetric target distance and an asymmetric reconstructed distance, but we were still comparing them through a forcibly symmetrized membership weight μ_{sym}. That is the inconsistency: the one piece of the pipeline that was not allowed to be directional was exactly the piece deciding how strongly each direction should be pulled or pushed.

1.3 The fix

stop symmetrizing the force weight. $A(i, j)$, the raw, one-sided membership described above, was already being computed. The only change needed was to use A itself in the force computation instead of μ_{sym}:

$$\text{force}(i, j) = A(i, j) \cdot \text{attr\_coeff}(\rho_r) \; - \; \big(1 - A(i, j)\big) \cdot \text{rep\_coeff}(\rho_r).$$

Now the cross-entropy sum runs over *ordered* pairs (i, j), each with its own directed weight, instead of over undirected edges sharing one value. This is directionally consistent with both D_{asym} and ρ_r. At this point, μ_{sym} was kept around for exactly one remaining job: feeding the initial layout, since that step's eigendecomposition still needed a symmetric input.

1.4 Result

There is now exactly one membership matrix in the pipeline, A, and it is never symmetrized. It is used only where the cross-entropy forces need it; initialization is built directly from the distance matrix and does not touch A at all. The asymmetric target distance, the asymmetric reconstructed distance, and the directed force weighting are now consistent with each other end to end.

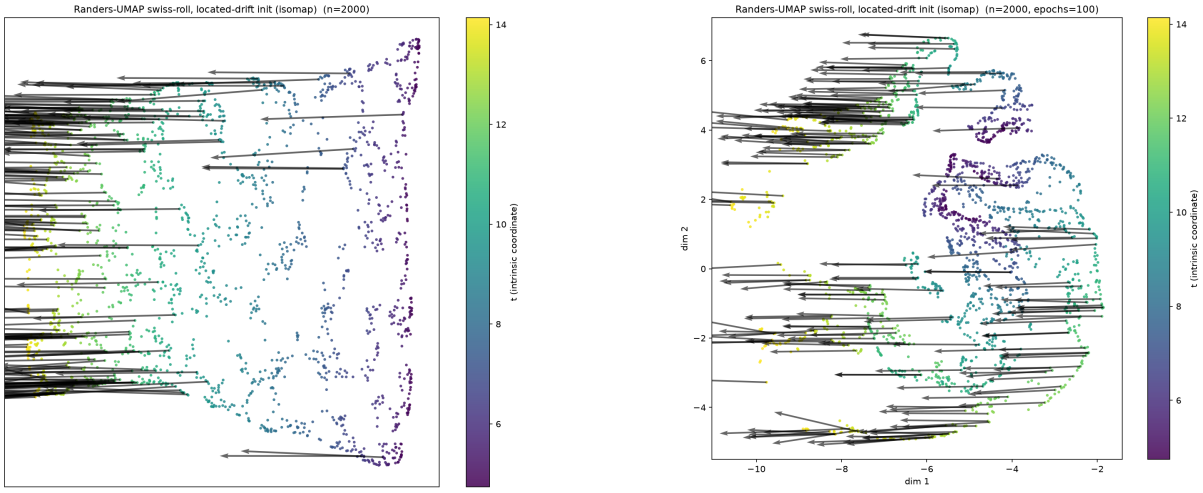

Figure 1: Swiss Roll Embedding before and after the fix

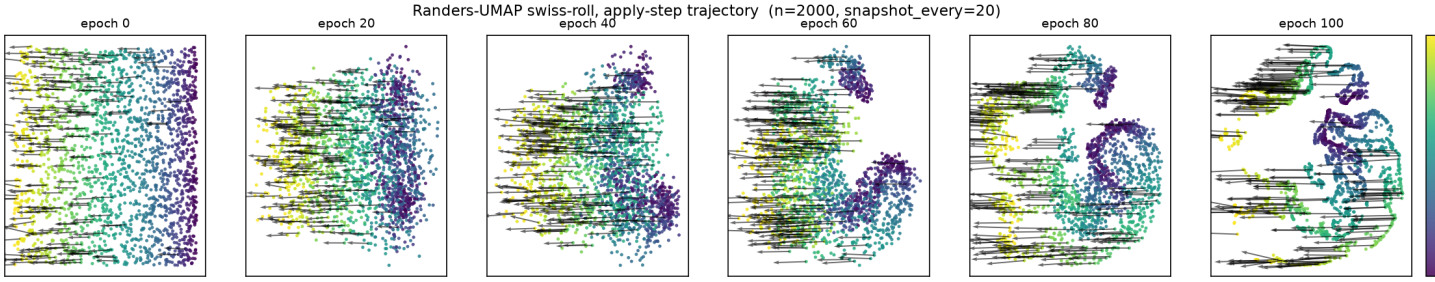

Figure 2: Swiss Roll Embedding Snapshots after the fix

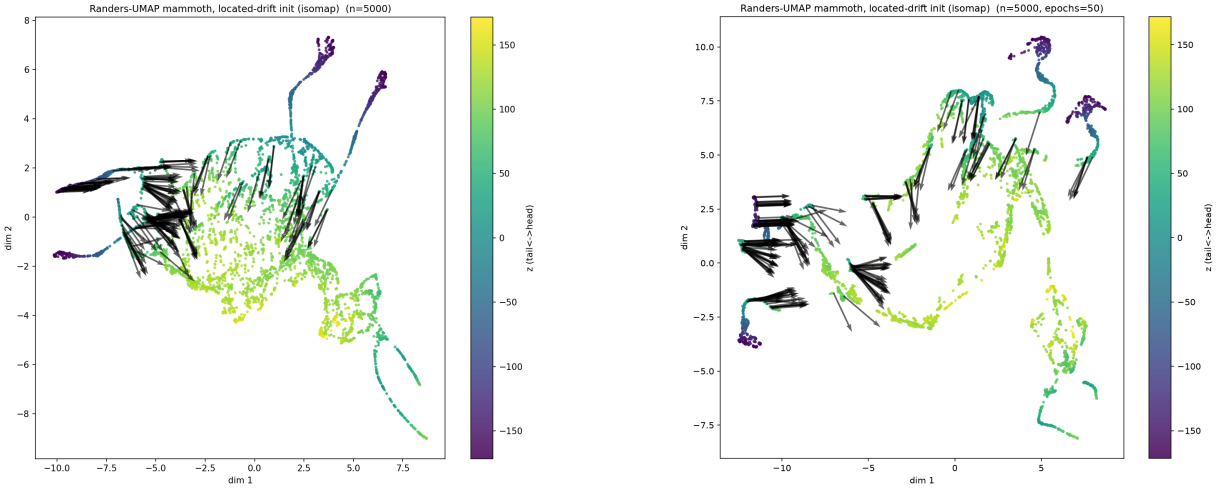

Figure 3: Mammoth Embedding before and after the fix

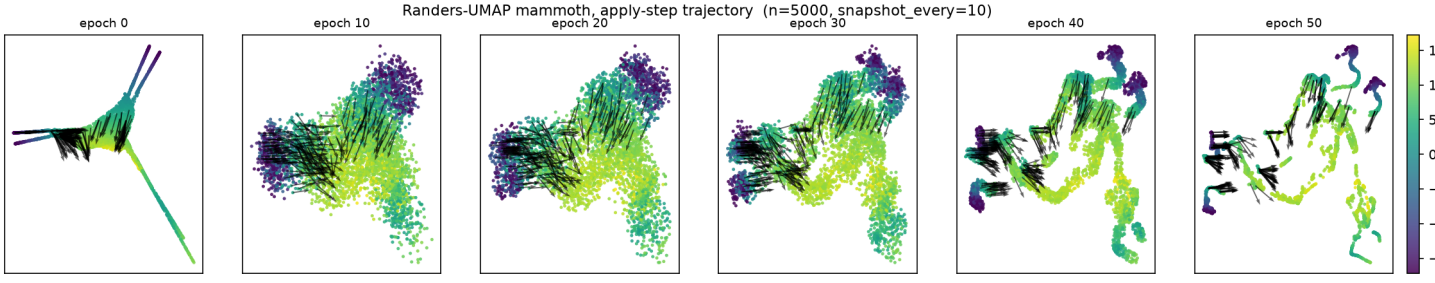

Figure 4: Mammoth Embedding Snapshots after the fix

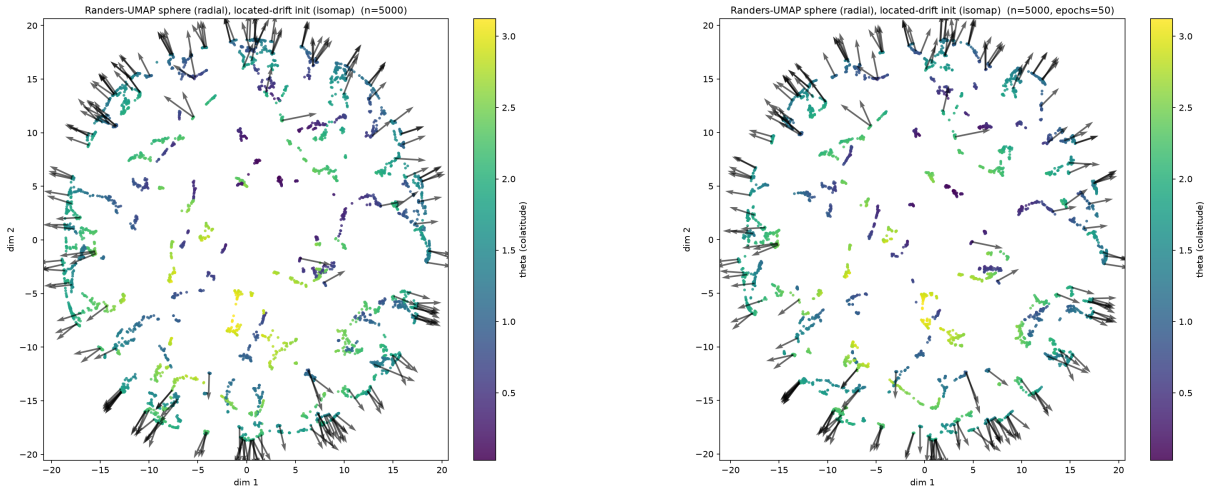

Figure 5: Sphere with Radial Drifts Embedding before and after the fix

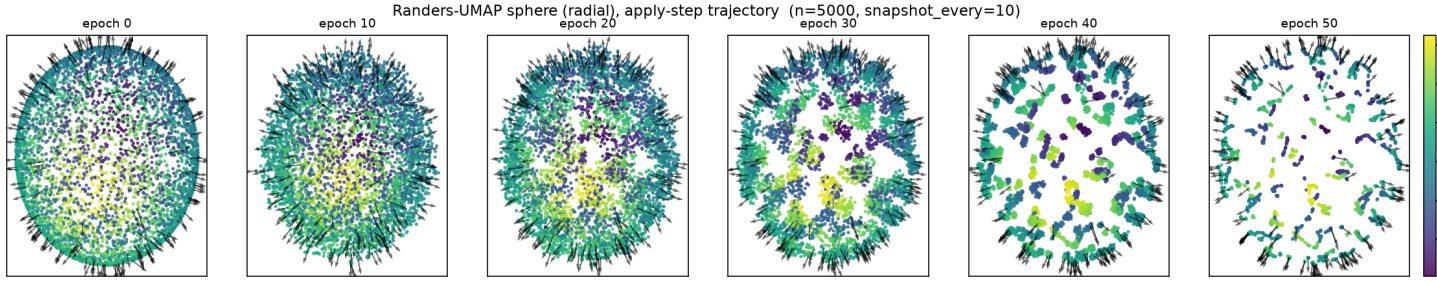

Figure 6: Sphere with Radial Drifts Embedding Snapshots after the fix

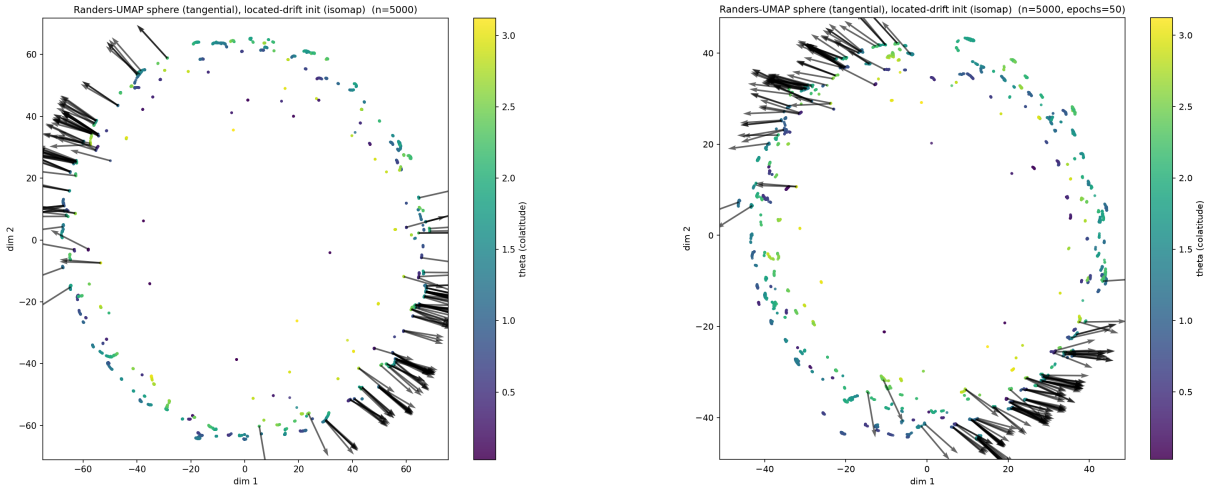

Figure 7: Sphere with Tangential Drifts Embedding before and after the fix

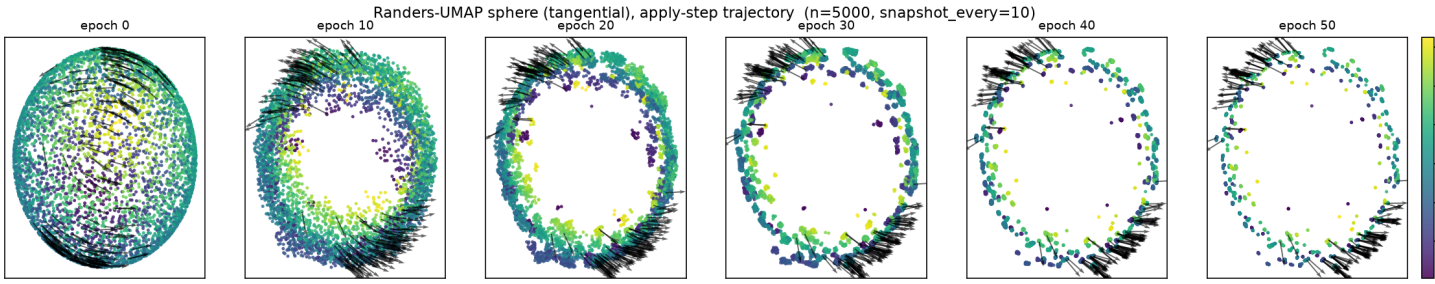

Figure 8: Sphere with Tangential Drifts Embedding Snapshots after the fix

2 Per-Node Alignment: Rank Consistency of Asymmetry Preservation

Alongside the global `asymmetry_score` (the ratio between the mean target and mean reconstructed asymmetry, one number per k), we also track a second, complementary diagnostic at each k: the *alignment* between the two per-node asymmetry vectors.

For every node i, two quantities are available: its target (initial) asymmetry, α_i, computed from D_{asym} before training, and its final asymmetry, $\hat{\alpha}_i$, computed from the trained embedding's reconstructed distances. The global `asymmetry_score` summarizes these as a single ratio of means, $\overline{\hat{\alpha}}/\overline{\alpha}$, which answers *how much* asymmetry survives training on average – but it says nothing about *which* nodes that surviving asymmetry belongs to. Two very different outcomes can produce the same ratio: training could preserve each node's asymmetry roughly proportionally (the nodes that started most asymmetric are still the most asymmetric), or it could redistribute asymmetry arbitrarily across nodes while keeping the same overall average.

To distinguish these cases, we compute the Pearson correlation between the two per-node vectors:

$$\text{alignment} = \frac{\sum_i (\alpha_i - \bar{\alpha})(\hat{\alpha}_i - \bar{\hat{\alpha}})}{\sqrt{\sum_i (\alpha_i - \bar{\alpha})^2}\,\sqrt{\sum_i (\hat{\alpha}_i - \bar{\hat{\alpha}})^2}} \in [-1, 1].$$

Nodes with no direct neighbours (isolated nodes, for which α_i or $\hat{\alpha}_i$ is undefined) are excluded before this computation. If fewer than two valid nodes remain, or if either vector is constant (zero variance, making the correlation undefined), alignment is reported as undefined (`nan`) rather than a misleading value.

An alignment near $+1$ means the ranking of nodes by asymmetry is preserved end to end: nodes that were most directionally asymmetric in the input remain the most asymmetric in the trained embedding, even if their absolute magnitude shrank. An alignment near 0 means no such relationship survives – the final asymmetry is effectively decoupled from where it originally came from. A negative alignment would indicate an inverted relationship, where the most-asymmetric input nodes become the least-asymmetric output nodes.

This is measured separately from, and answers a different question than, the magnitude-based `asymmetry_score`: the latter measures *how much* is preserved on average; alignment measures *whether the preservation is structured* rather than arbitrarily scattered across nodes.

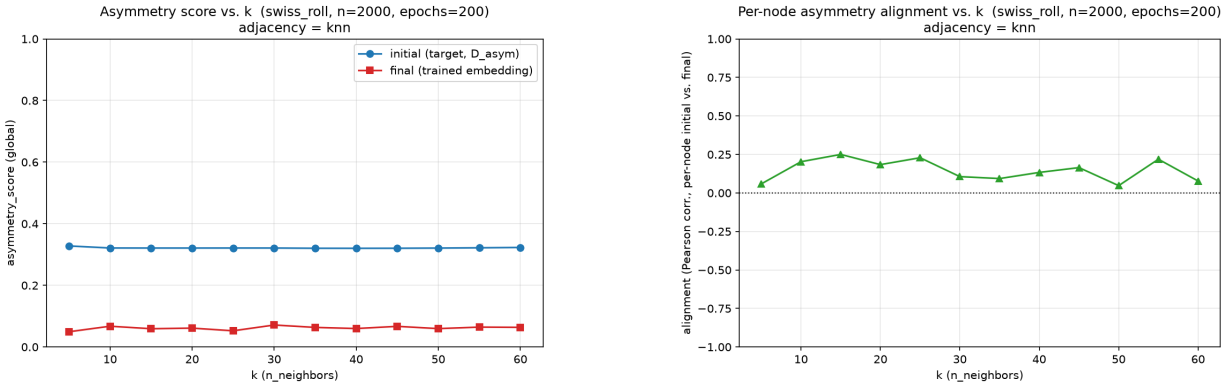

Figure 9: Swiss Roll Results

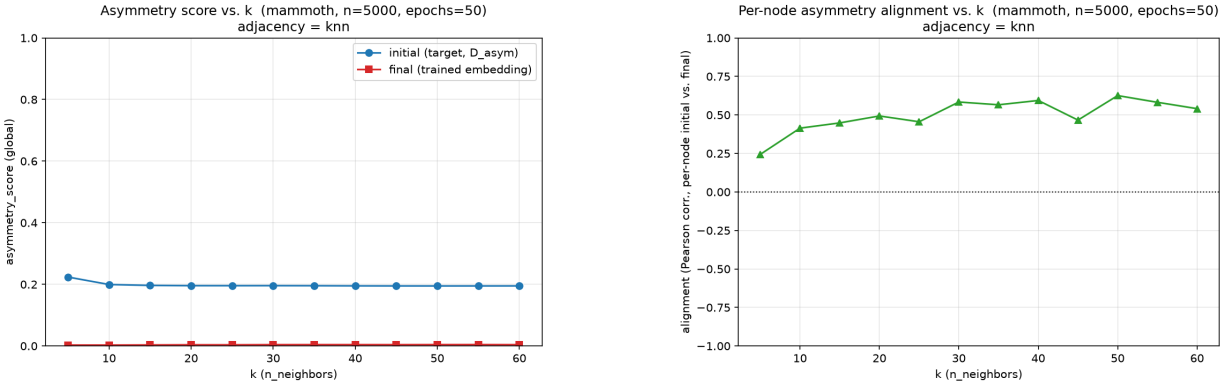

Figure 10: Mammoth Results

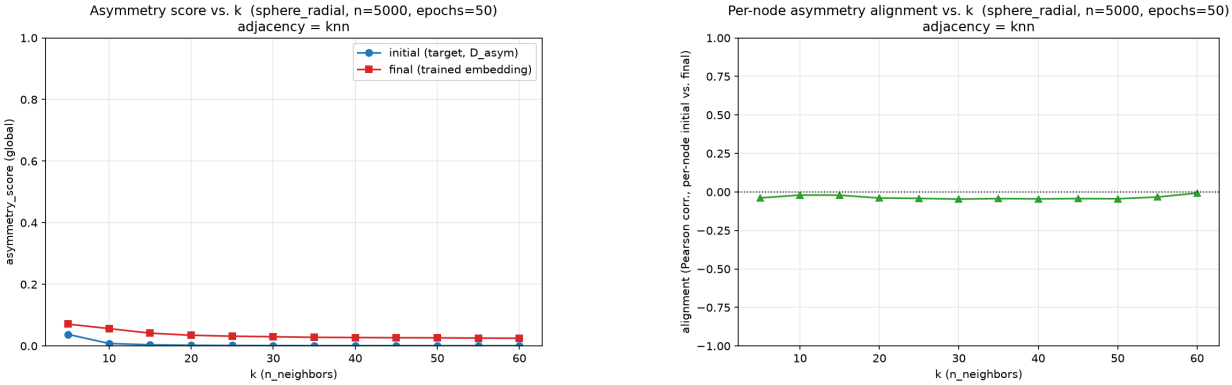

Figure 11: Sphere with Radial Drifts Results

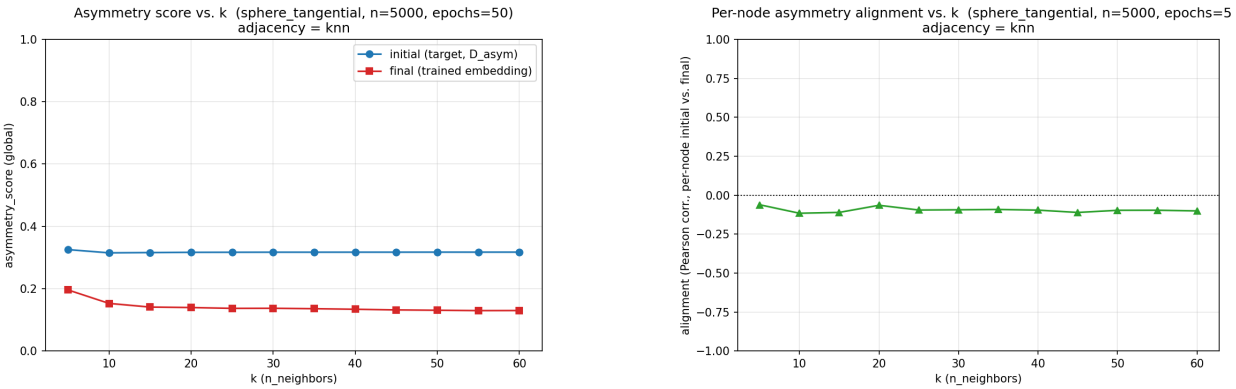

Figure 12: Sphere with Tangential Drifts Results

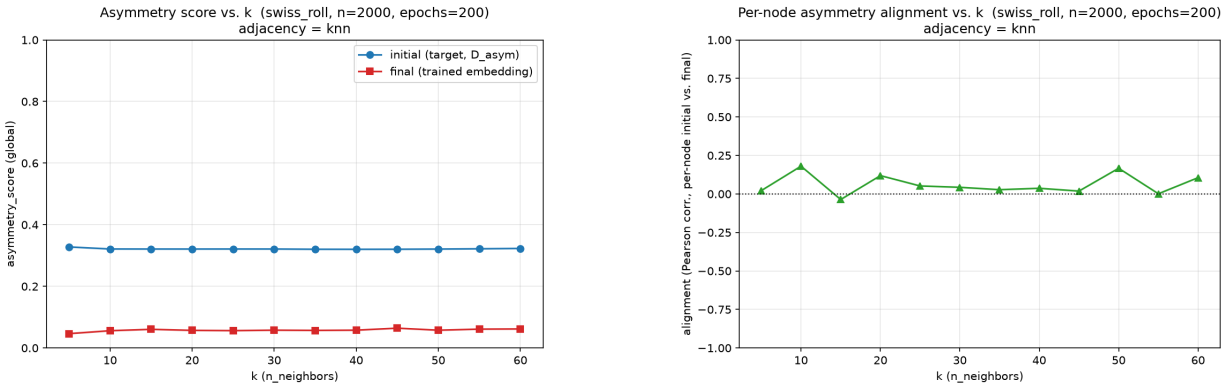

Figure 13: Swiss Roll Results after the fix

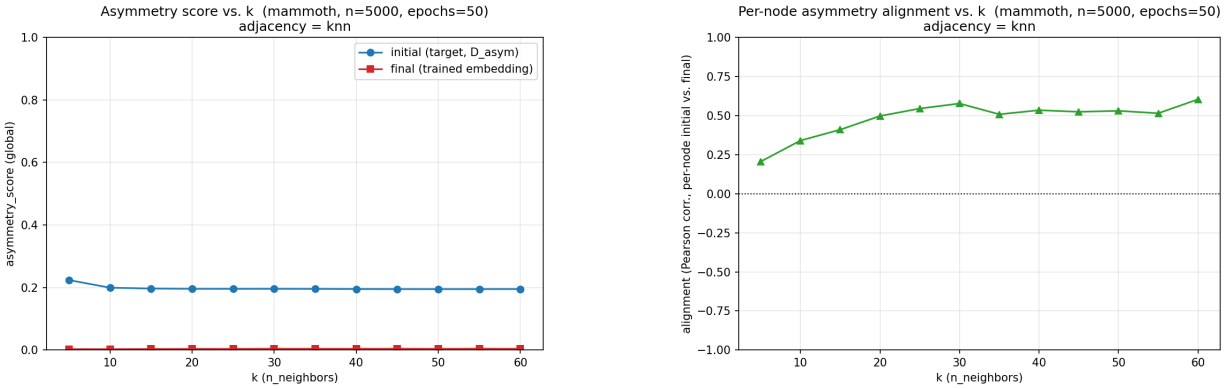

Figure 14: Mammoth Results after the fix

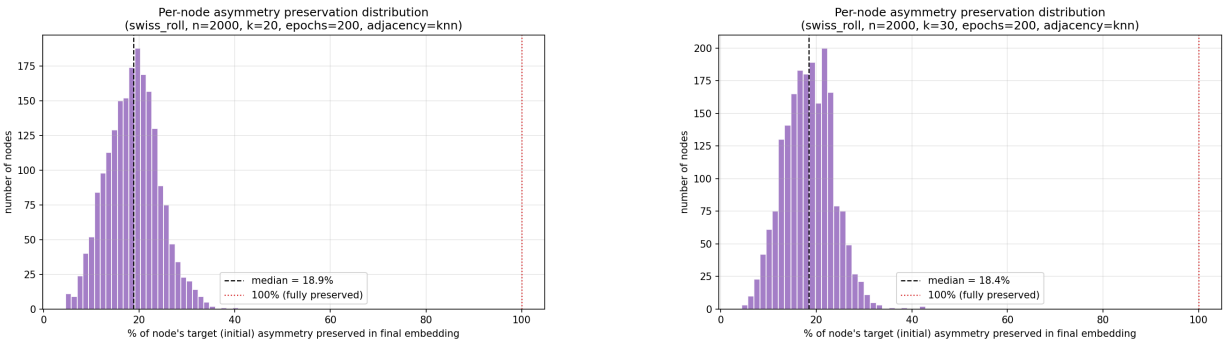

Figure 15: Asymmetry Distributions of Swiss Roll for k=20 and k=30